

Two Pointers

Basics → Master: Full Theory + LeetCode Problem Bank

Same struggle-first rule as before: attempt each problem cold (20-30 min) before reading the approach notes here. Two Pointers looks simple on the surface but has four genuinely different sub-patterns — confusing them is the main reason people plateau on this topic. This guide separates them clearly so you know which one you're using and why.

Section	Covers
1. What & Why	Core idea, how it differs from sliding window
2. Four Core Sub-Patterns	Opposite-direction, same-direction, fast-slow, merge-style
3. Java Templates	Reusable skeleton for each sub-pattern
4. Problem Bank (Tiers 1-6)	28 curated LeetCode problems, basics → master
5. Fully Worked Examples	4 problems solved with full reasoning + code
6. Fast & Slow Pointer Theory	Cycle detection, middle-of-list, Floyd's algorithm
7. Common Mistakes	Wrong pointer direction, unsorted-array traps, etc.
8. Practice Roadmap	Order to solve + spaced repetition schedule

1. What is Two Pointers & Why It Works

Two Pointers uses two indices moving through a data structure — usually an array or linked list — to avoid a nested loop. Instead of checking every pair (i, j) which costs $O(n^2)$, you move two pointers based on a comparison rule so each pointer only travels forward (or one forward, one backward) once. Total work becomes $O(n)$ or $O(n \log n)$ if sorting is needed first.

How this differs from Sliding Window

They look similar (both use left/right indices) but solve different problems:

- **Sliding Window** always keeps a *contiguous* range $[left, right]$ and cares about the **window itself** (its sum, its content).
- **Two Pointers** often has pointers that **don't** define a contiguous range at all — e.g. one starts at index 0, the other at index $n-1$, moving toward each other. The pointers represent two *candidates* being compared, not the edges of a window.
- Rule of thumb: if the two pointers ever move *toward each other* or represent *two separate positions being compared* (not a shrinking/growing contiguous block), it's Two Pointers, not Sliding Window.

How to recognize a Two Pointers problem

- The input is **sorted** (or can be sorted) and you need pairs/triplets matching a sum/condition.
- You need to **partition or rearrange** an array in-place (move zeros, Dutch national flag, remove duplicates).
- You're comparing from **both ends** inward (palindrome check, container with most water, trapping rain water).
- You need to detect a **cycle** or find the **middle** of a linked list (fast-slow variant).
- Brute force would be $O(n^2)$ checking every pair — that's your signal to ask "can two pointers replace this nested loop?"

Key prerequisite most people skip: the opposite-direction and same-direction patterns almost always need the array **sorted first**. If you're stuck, ask: "would sorting make a greedy pointer-move decision obvious?" That question unlocks most Tier 1-3 problems.

2. The Four Core Sub-Patterns

2.1 Opposite-Direction Pointers (converging)

One pointer starts at index 0, the other at index $n-1$. They move toward each other based on a comparison, until they meet. Requires the array to be sorted (for sum-based problems) or works directly on symmetric structures (palindrome check).

Theory

- If sum/comparison at (left, right) is too small $\rightarrow$ move left forward (increase value, since array is sorted ascending).
- If too large $\rightarrow$ move right backward (decrease value).
- If it matches $\rightarrow$ record answer, then move both (or one) inward to continue searching.
- Correctness relies on: moving the 'wrong' pointer can never produce a better/valid answer, because of the sorted order — this is the greedy proof you should be able to state out loud.

2.2 Same-Direction Pointers (slow-fast on array, not linked list)

Both pointers start at index 0. One (fast/reader) scans ahead; the other (slow/writer) only advances when a condition is met. Used for in-place array modification.

Theory

- fast explores every element once — $O(n)$ traversal.
- slow marks the 'next position to write a valid element to' — it only moves when fast finds something that should be kept.
- At the end, slow (or slow+1) is the new valid length of the modified array.

2.3 Fast-Slow Pointers (Floyd's cycle / linked list)

Both pointers start at the head of a linked list. fast moves 2 steps for every 1 step of slow. Used to find the middle of a list, detect cycles, or find a cycle's starting node.

Theory

- If there's a cycle, fast will eventually 'lap' slow and they meet inside the cycle — this is a mathematical certainty, not a heuristic (relative speed of 1 step/iteration guarantees a catch-up within one lap).
- If there's no cycle, fast reaches null first — safe termination condition.
- When fast reaches the end in a list of length n , slow is exactly at the middle (this exploits the 2:1 speed ratio directly).

2.4 Merge-Style Pointers (two separate sorted arrays/lists)

One pointer per array/list, both starting at their own beginning, advancing based on comparison between the two current elements. This is the backbone of merge sort's merge step and any "merge two sorted X" problem.

Theory

- Compare current elements of both structures; advance the pointer whose element was smaller (or consumed).
- One structure may run out first — remember to append the leftover tail of the other.

- This pattern generalizes to k-way merges using a heap instead of plain pointers (see Tier 6).

3. Java Templates (memorize the shape, not the code)

3.1 Opposite-direction template

```
// Two Sum on a SORTED array
public int[] twoSumSorted(int[] nums, int target) {
    int left = 0, right = nums.length - 1;

    while (left < right) {
        int sum = nums[left] + nums[right];
        if (sum == target) return new int[]{left, right};
        else if (sum < target) left++; // need a bigger sum -> move left up
        else right--; // need a smaller sum -> move right down
    }
    return new int[]{-1, -1};
}
```

3.2 Same-direction (in-place write) template

```
// Remove duplicates from sorted array, in place
public int removeDuplicates(int[] nums) {
    int slow = 0; // next write position

    for (int fast = 1; fast < nums.length; fast++) {
        if (nums[fast] != nums[slow]) {
            slow++;
            nums[slow] = nums[fast];
        }
    }
    return slow + 1; // new length
}
```

3.3 Fast-slow (linked list) template

```
// Detect cycle in a linked list (Floyd's algorithm)
public boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true; // they met -> cycle exists
    }
    return false; // fast hit null -> no cycle
}
```

3.4 Merge-style template

```
// Merge two sorted arrays into a new array
public int[] mergeSorted(int[] a, int[] b) {
    int i = 0, j = 0, k = 0;
    int[] result = new int[a.length + b.length];

    while (i < a.length && j < b.length) {
        result[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
    }
    while (i < a.length) result[k++] = a[i++];
    while (j < b.length) result[k++] = b[j++];
    return result;
}
```

Four templates, four different pointer-movement rules. The skill isn't memorizing these — it's correctly identifying WHICH of the four a new problem needs before you start coding.

4. Problem Bank — Tier 1 to Tier 6 (28 problems)

Solve in order. Tiers 1-2 build the opposite/same-direction reflex; Tier 3 is fast-slow on linked lists; don't jump to Tier 5-6 before that reflex is automatic.

Tier 1 — Opposite-Direction Basics

LC#	Problem	Diff.	Key Concept / Theory
167	Two Sum II - Input Array Is Sorted	Med	Pure opposite-direction template; the canonical starting problem.
125	Valid Palindrome	Easy	Opposite pointers comparing chars from both ends, skipping non-alphanumeric.
11	Container With Most Water	Med	Move the pointer with the SMALLER height inward — moving the taller one can never improve area (key greedy proof to internalize).
42	Trapping Rain Water	Hard	Opposite pointers + track leftMax/rightMax; water at each position bound by the smaller of the two maxes.
977	Squares of a Sorted Array	Easy	Opposite pointers comparing absolute values, filling result array from the back.

Tier 2 — Same-Direction / In-Place Rearrangement

LC#	Problem	Diff.	Key Concept / Theory
26	Remove Duplicates from Sorted Array	Easy	Classic slow=write-pointer, fast=read-pointer template.
283	Move Zeroes	Easy	Same template — slow marks next non-zero write position, then fill remainder with zeros.
27	Remove Element	Easy	Same-direction pointer, condition is 'not equal to val' instead of 'not duplicate'.
75	Sort Colors (Dutch National Flag)	Med	THREE pointers: low, mid, high — partitions array into 3 regions in one pass. Extension of same-direction idea.
80	Remove Duplicates from Sorted Array II	Med	Same-direction, but allow up to 2 duplicates — compare fast to slow-2 instead of slow.

Tier 3 — Fast-Slow Pointers (Linked List)

LC#	Problem	Diff.	Key Concept / Theory
876	Middle of the Linked List	Easy	fast moves 2x speed; when fast hits end, slow is at the middle — direct application of the 2:1 ratio.
141	Linked List Cycle	Easy	Floyd's algorithm — if slow and fast ever meet, a cycle exists.
142	Linked List Cycle II (find cycle start)	Med	After slow/fast meet, reset one pointer to head; move both 1 step at a time — they meet exactly at the cycle's start (provable with the math of the meeting-point distance).

LC#	Problem	Diff.	Key Concept / Theory
234	Palindrome Linked List	Easy	Fast-slow to find middle, reverse second half, then opposite-direction-style compare from both ends.
143	Reorder List	Med	Fast-slow to find middle + reverse second half + merge-style interleaving — combines three sub-patterns.

Tier 4 — Two Pointers + Sorting (triplets/quadruplets)

LC#	Problem	Diff.	Key Concept / Theory
15	3Sum	Med	Sort array, fix one element, run opposite-direction two-pointer on the rest — skip duplicates carefully at both the fixed index and inside the two-pointer loop.
16	3Sum Closest	Med	Same as 15, but track closest sum-to-target difference instead of exact matches.
18	4Sum	Med	Fix two elements (nested loop), then opposite-direction two-pointer on the remaining two — generalizes 3Sum by one more fixed layer.
259	3Sum Smaller	Med	Opposite-direction two-pointer, but count ALL valid pairs at once when sum < target (right - left pairs), not just find one.
611	Valid Triangle Number	Med	Sort, fix the largest side, two-pointer on the other two — count pairs satisfying triangle inequality in bulk, similar counting trick to 259.

Tier 5 — Merge-Style & Partition Variants

LC#	Problem	Diff.	Key Concept / Theory
88	Merge Sorted Array	Easy	Merge-style but IN PLACE — fill from the back to avoid overwriting unprocessed elements (reverse merge direction trick).
21	Merge Two Sorted Lists	Easy	Merge-style template directly on linked list nodes instead of arrays.
986	Interval List Intersections	Med	Merge-style pointer per interval list; advance whichever interval ends first.
532	K-diff Pairs in an Array	Med	Sort + same-direction pointer skipping duplicates, distinct from opposite-direction because k can be 0.

Tier 6 — Master Tier (combined techniques)

LC#	Problem	Diff.	Key Concept / Theory
407	Trapping Rain Water II (2D)	Hard	Extends Tier 1's trapping logic to a grid using a min-heap boundary instead of two simple pointers — shows how the core idea generalizes beyond 1D.
881	Boats to Save People	Med	Sort + opposite-direction greedy pairing (lightest with heaviest) — a disguised two-pointer greedy, easy to miss.

LC#	Problem	Diff.	Key Concept / Theory
977	(Revisit) Squares of Sorted Array	Easy	Revisit after Tier 4-5 — notice the fill-from-back trick reappears in 88 and here, a transferable sub-skill.
25	Reverse Nodes in k-Group	Hard	Combine fast-slow-style group boundary detection with pointer-based in-place reversal — strong integration problem before moving to the next pattern.

5. Fully Worked Examples (with reasoning)

Example A — 11. Container With Most Water (opposite-direction greedy proof)

Problem: Given heights, find two lines that with the x-axis form a container holding the most water.

Reasoning

- Start with $\text{left}=0$, $\text{right}=n-1$ — the widest possible container.
- $\text{Area} = \text{width} * \min(\text{height}[\text{left}], \text{height}[\text{right}])$. Width only shrinks as pointers move inward, so the ONLY way to potentially increase area is if the new min height is bigger.
- Since the shorter line is always the bottleneck, moving the TALLER pointer can never help (width shrinks, and the min height can only shrink or stay the same because the shorter side is still there). So always move the SHORTER pointer inward.

```
public int maxArea(int[] height) {
    int left = 0, right = height.length - 1, maxArea = 0;

    while (left < right) {
        int width = right - left;
        int minHeight = Math.min(height[left], height[right]);
        maxArea = Math.max(maxArea, width * minHeight);

        if (height[left] < height[right]) left++;    // shorter side moves
        else right--;
    }
    return maxArea;
}
```

Example B — 15. 3Sum (sorted + fixed element + opposite-direction)

Problem: Find all unique triplets in the array that sum to zero.

Reasoning

- Sort the array first — this unlocks both duplicate-skipping and the opposite-direction two-pointer.
- Fix $\text{nums}[i]$ as the first element (outer loop), then run opposite-direction two-pointer on the remaining subarray to find pairs that sum to $-\text{nums}[i]$.
- Skip duplicate values at BOTH the outer index and inside the two-pointer loop to avoid duplicate triplets in the output.

```
public List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> result = new ArrayList<>();

    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue; // skip duplicate fixed element

        int left = i + 1, right = nums.length - 1;
        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                result.add(Arrays.asList(nums[i], nums[left], nums[right]));
                while (left < right && nums[left] == nums[left + 1]) left++; // skip dup
                while (left < right && nums[right] == nums[right - 1]) right--; // skip dup
                left++; right--;
            } else if (sum < 0) left++;
            else right--;
        }
    }
    return result;
}
```

```

    }
}
return result;
}

```

Example C — 142. Linked List Cycle II (fast-slow, find cycle start)

Problem: If a linked list has a cycle, return the node where the cycle begins.

Reasoning — the math, not just the trick

- Let distance from head to cycle start = a, cycle start to meeting point = b, remaining cycle length = c.
- When slow and fast meet: slow traveled a+b, fast traveled a+b+c+b (one extra full lap). Since fast = 2*slow: $2(a+b) = a+2b+c$, which simplifies to $a = c$.
- This means: distance from head to cycle-start EQUALS distance from meeting-point to cycle-start (going forward). So reset one pointer to head, keep the other at the meeting point, move both one step at a time — they meet exactly at the cycle start.

```

public ListNode detectCycle(ListNode head) {
    ListNode slow = head, fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) {                // cycle found, meeting point
            ListNode ptr = head;
            while (ptr != slow) {           // move both 1 step -> meet at cycle start
                ptr = ptr.next;
                slow = slow.next;
            }
            return ptr;
        }
    }
    return null; // no cycle
}

```

Example D — 75. Sort Colors (Dutch National Flag, 3-pointer partition)

Problem: Sort an array of 0s, 1s, and 2s in place, in one pass.

Reasoning

- Three pointers: low (boundary for 0s), mid (current element), high (boundary for 2s).
- If `nums[mid] == 0`: swap with low, advance both low and mid (safe because the swapped-in value at mid was already known to be at most 1 from low's region).
- If `nums[mid] == 1`: just advance mid (already in correct middle region).
- If `nums[mid] == 2`: swap with high, advance high ONLY (don't advance mid — the swapped-in value from high is unknown and must be re-checked).

```

public void sortColors(int[] nums) {
    int low = 0, mid = 0, high = nums.length - 1;

    while (mid <= high) {
        if (nums[mid] == 0) {
            swap(nums, low++, mid++);
        } else if (nums[mid] == 1) {
            mid++;
        }
    }
}

```

```
        } else { // nums[mid] == 2
            swap(nums, mid, high--); // do NOT increment mid here
        }
    }
}
private void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }
```

6. Fast & Slow Pointer Theory (deep dive)

This sub-pattern deserves its own section because it's the one most people apply mechanically without understanding why it's guaranteed to work.

Why fast-slow guarantees a meeting inside a cycle

- Once slow enters the cycle, think of the gap between fast and slow (in steps around the cycle) as shrinking by exactly 1 each iteration (fast gains 2 steps, slow gains 1, net gain 1 per iteration).
- A gap that shrinks by 1 every iteration and starts less than the cycle length MUST hit 0 within one full lap — this is why they're guaranteed to meet, not just 'usually' meet.

Why fast reaching the end means slow is at the middle

- fast covers 2 nodes for every 1 node slow covers. If the list has n nodes, when fast has covered n (or $n-1$) nodes, slow has covered $n/2$ — the exact middle.
- For even-length lists, this naturally gives the second middle node — adjust the loop condition (`fast != null` vs `fast.next != null`) depending on whether you want the first or second middle.

If you can derive the $a = c$ equation from Example C yourself without looking, you've actually understood fast-slow — not just memorized the reset-to-head trick.

7. Common Mistakes

- **Forgetting to sort first:** opposite-direction and 3Sum/4Sum style problems silently fail on unsorted input — the greedy pointer-move logic depends entirely on sorted order.
- **Moving the wrong pointer in Container With Most Water-style problems:** always move the pointer with the SMALLER value — moving the taller one can only ever hurt or maintain, never help.
- **Off-by-one in fast-slow loop conditions:** `fast != null and fast.next != null` vs `fast != null` changes which middle you land on for even-length lists — decide on purpose, not by accident.
- **Not skipping duplicates in 3Sum/4Sum:** forgetting to skip duplicate values at the fixed index AND inside the two-pointer loop produces duplicate triplets in the output.
- **Advancing mid after a swap-with-high in Dutch Flag:** the swapped-in value from the high region is unverified and must be re-examined — advancing mid here is a classic bug.
- **Using two pointers on a problem that's actually sliding window:** if you need to track a CONTIGUOUS range's aggregate (sum, count), that's sliding window; if you're comparing two specific candidate positions, that's two pointers. Don't force the wrong template.

8. Practice Roadmap & Spaced Repetition

Suggested order (2-3 problems/day)

- **Days 1-3:** Tier 1 (all 5 problems) — opposite-direction reflex + the Container/Trapping greedy proofs.
- **Days 4-6:** Tier 2 (all 5 problems) — same-direction in-place rearrangement, ending with Dutch National Flag's 3-pointer extension.
- **Days 7-10:** Tier 3 (all 5 problems) — fast-slow on linked lists; derive the $a=c$ proof yourself for problem 142 before reading the answer.
- **Days 11-14:** Tier 4 (all 5 problems) — sorted-array + fixed-element combos (3Sum family).
- **Days 15-17:** Tier 5 (all 4 problems) — merge-style, including the in-place reverse-fill trick in 88.
- **Days 18-21:** Tier 6 — attempt cold, 30-40 min each, these combine multiple sub-patterns in one problem.

Spaced repetition after first solve

- Day 1: solve with full struggle (aim 25-30 min before hints).
- Day 3: re-solve cold — check whether you correctly identify WHICH of the 4 sub-patterns applies before writing any code.
- Day 7: re-solve cold again; write one line on the hardest sub-decision (which pointer to move? sort first? fast-slow ratio?).
- Day 21: final cold re-solve — fast completion here means recall, not recognition.

Same method as the Sliding Window guide — this is a reference to check AFTER attempting, not a shortcut around the struggle.